

Assignment 2

April 09, 2026

SHAO Junjie | 314551819

GitHub link: <https://github.com/2ec3/DM2026-Assignment-1.git>

1. K-fold Cross-Validation (15 points):

For Question 1, please continue using `Real_World_Classification.ipynb`.

Before starting:

- First, please update `linear_model.py` by downloading or replacing it with the latest version from https://github.com/NYCU-Data-Mining/DM2026-Assignment-1/blob/main/model/linear_model.py.
- For this question, please use your preprocessed data from Assignment 1, Question 3.
- Do not change the `random_seed` to ensure consistency in evaluation results.

- a. K-fold Cross-Validation:** Use `scikit-learn` to implement 5-fold cross-validation on the training data to determine the optimal hyperparameters.
- Apply L2 regularization
 - Evaluate the following hyperparameter values (**16 combinations** in total)
 - Learning rates: {0.005, 0.01, 0.1, 0.5}
 - Regularization parameters (`reg_lambda` in the code): {1.0, 2.0, 4.0, 8.0}
 - For each combination, compute the **average accuracy** over the 5 folds on the **training data**, and present the results in a 4×4 table.

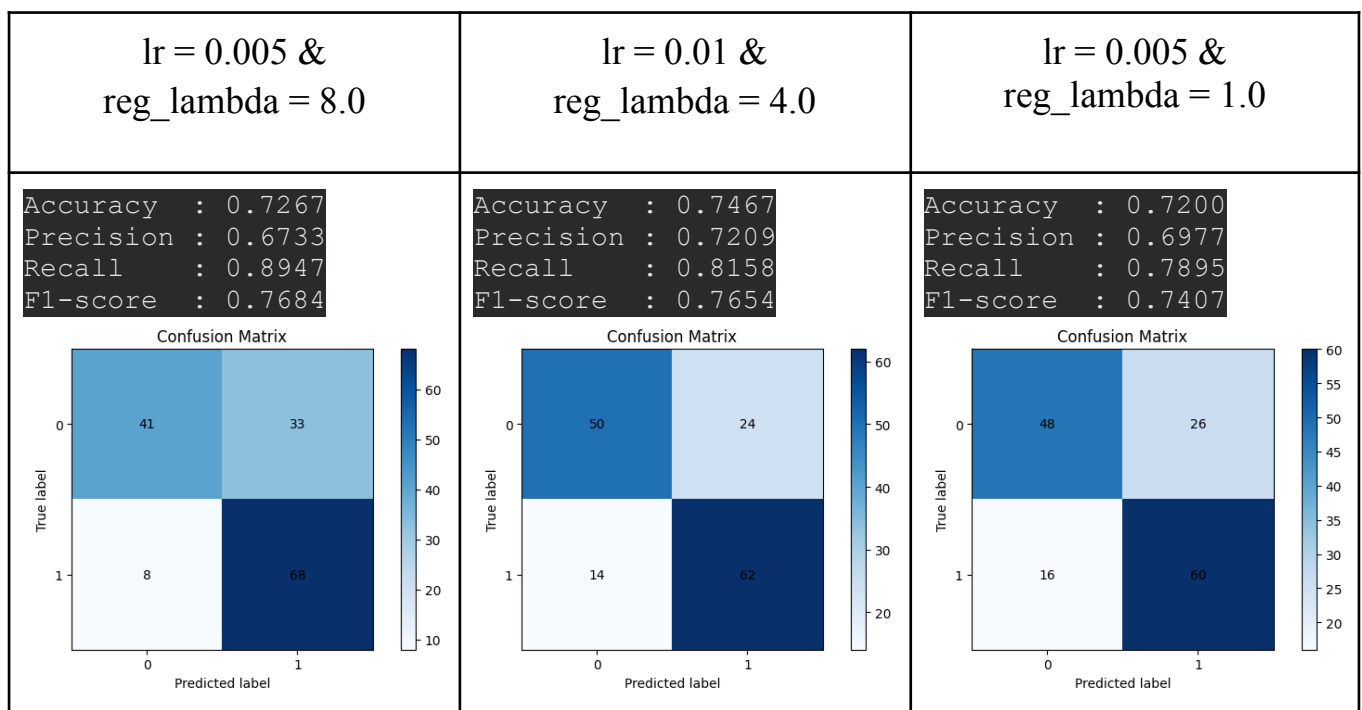
Average accuracy

reg_lambda \ lr	0.005	0.01	0.1	0.5
1.0	0.588571428571	0.665714285714	0.608571428571	0.642857142857
2.0	0.622857142857	0.642857142857	0.611428571428	0.622857142857

4.0	0.602857142857	0.68	0.62	0.654285714285
8.0	0.659999999999	0.651428571428	0.60285714285	0.611428571428

- b. **Following 1 (a)**, select the top two hyperparameter settings based on the training data (two hyperparameter combinations with the highest average accuracy). Use these two settings, train the model on the training data and evaluate it on the **testing data**. Report the **testing results** using the “**evaluate_binary_classifier**” function.

Top hyperparameter settings: (lr=0.005; reg_lambda=8.0) and (lr=0.01; reg_lambda=4.0)



- c. **Describe your observations in 1(a) and 1(b).**

I also included the testing results of the worst hyperparameter settings for better comparison.

We classified the hyperparameters according to the k-fold cross-validation method in 1(a), which has the goal of indicating the model is general (not overfitting the training data).

And with 1(b), we can clearly see that the top 2 hyperparameter settings have better testing results (I think that the most important result is the accuracy) than the worst hyperparameter settings.

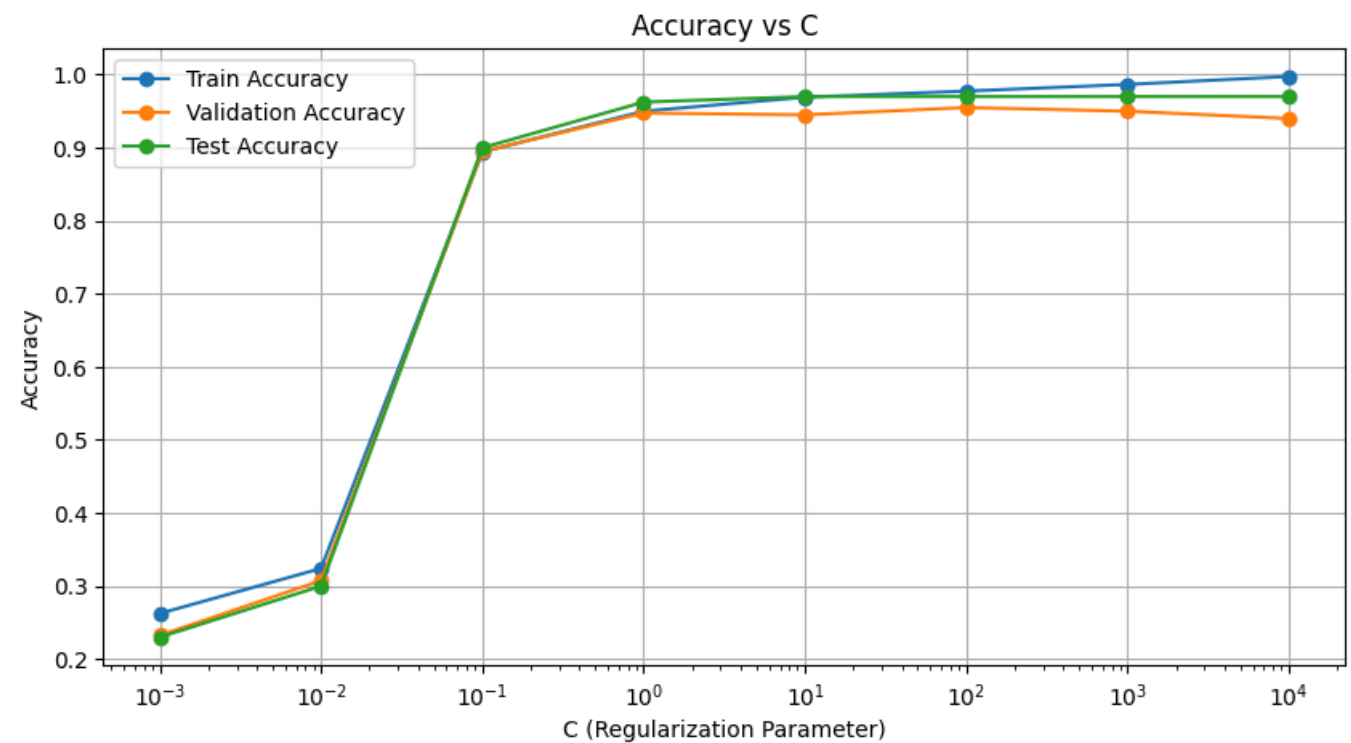
Thus, the testing results validate our method.

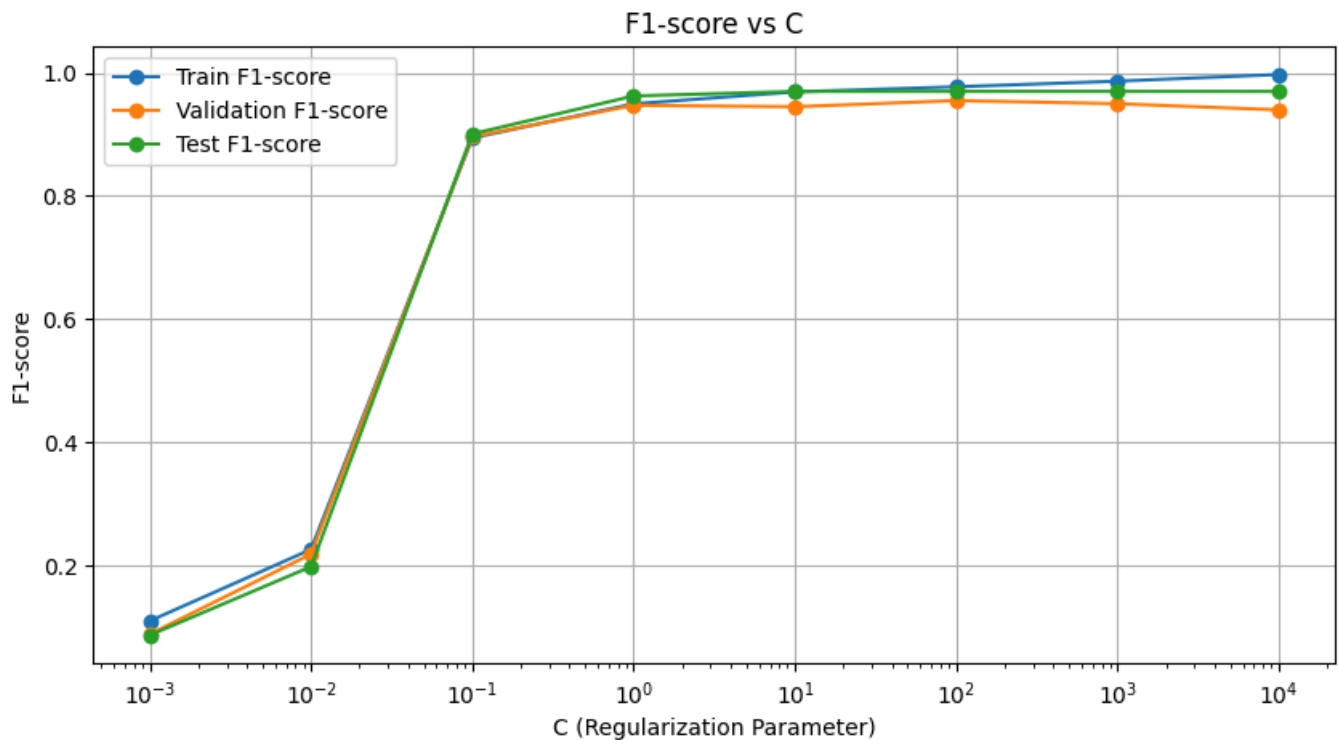
2. SVM (15 points):

- a. Shuffle the data (“mobile_price.csv”) using a random seed 42, and split it into training, validation, and testing sets with a ratio of 60% / 20% / 20% ratio. Then, train a SVM classifier with the regularization parameter $C=1.0$. Report the accuracy and F1-score on training, validation and testing data, respectively.

training	validation	testing
Accuracy : 0.9500 F1-score : 0.9498	Accuracy : 0.9475 F1-score : 0.9474	Accuracy : 0.9625 F1-score : 0.9626

- b. Explore different values of the regularization parameter $C = \{0.001, 0.01, 0.1, \dots, 100, 1000, 10000\}$. For each value of C , train the SVM model and evaluate its performance on the training, validation, and test sets using both accuracy and F1-score. Visualize the results to show how performance changes across different values of C .





- c. Following 2 (b), based on your observations, determine which model (which value of C) provides the best generalization performance, and explain your reasoning.

In my opinion, the order of importance for each dataset score is: test $\geq$ validation $>$ train. The accuracy / F1-score for the test dataset reaches its maximum at $C = 100$ (accuracy = 0.97; F1-score = 0.9701). Thus, I think that the value $C = 100$ provides the best generalization performance.

3. Association Rule Mining (15 points):

- a. Please list all frequent patterns showing support ≥ 0.3 using FP-growth.

support	itemsets
2 0.682	(ram_medium)
3 0.416	(px_width_medium)
5 0.414	(battery_power_medium)
4 0.412	(int_memory_medium)
7 0.318	(ram_medium, battery_power_medium)
0 0.316	(int_memory_low)
1 0.308	(battery_power_low)
6 0.306	(ram_medium, px_width_medium)

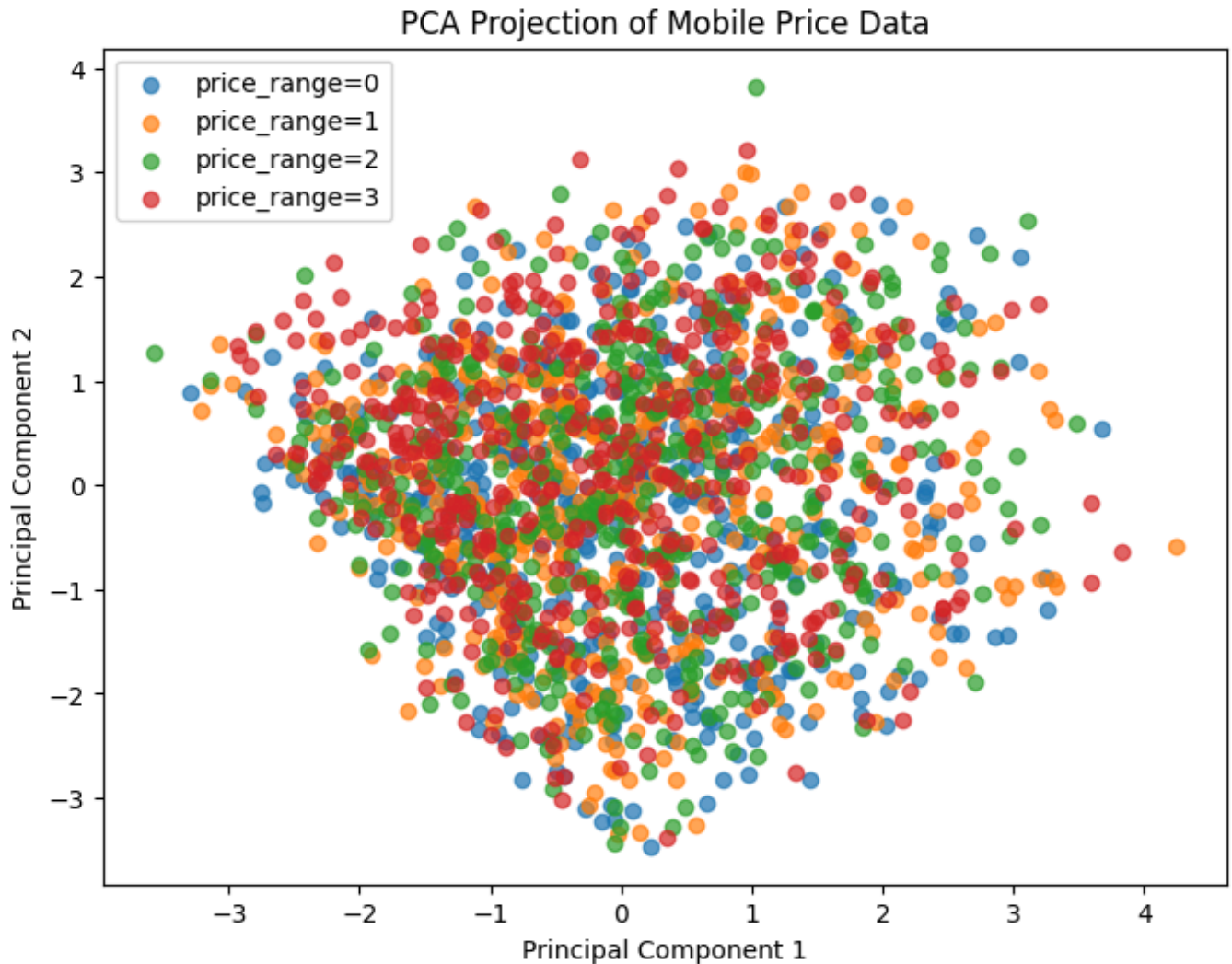
- b. Following 3(a), please list all the association rules where support ≥ 0.3 , confidence ≥ 0.4 and lift ≥ 0.8

	antecedents	consequents	support	confidence	lift
1	(battery_power_medium)	(ram_medium)	0.318	0.768116	1.126270

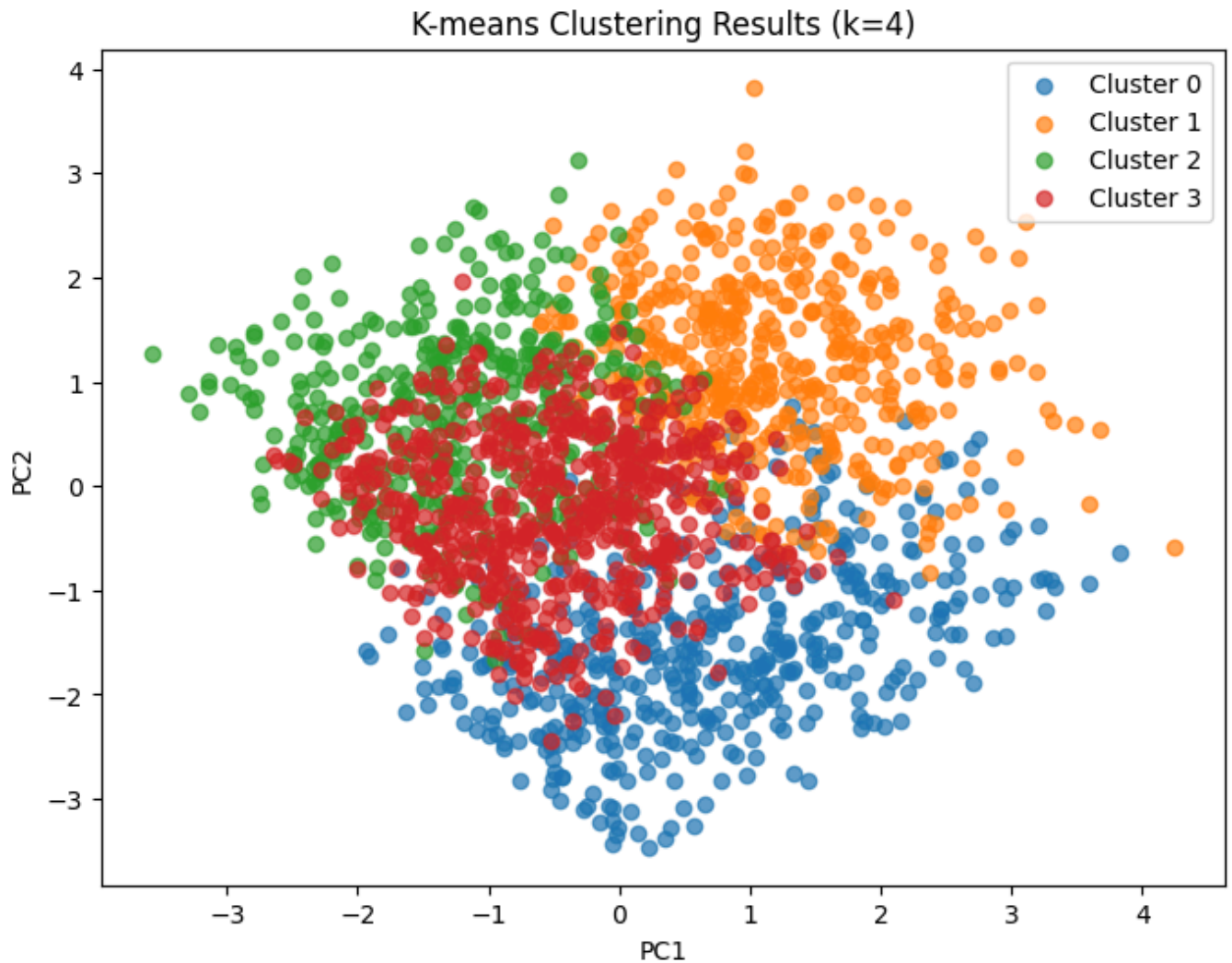
50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00	50e +00
------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------

We can see that the values are normalized (mean ≈ 0) and standardized (std ≈ 1).

- b. Following 4(a), project the features onto 2 dimensions using PCA and visualize the scatterplot for the first two principal components of the data.

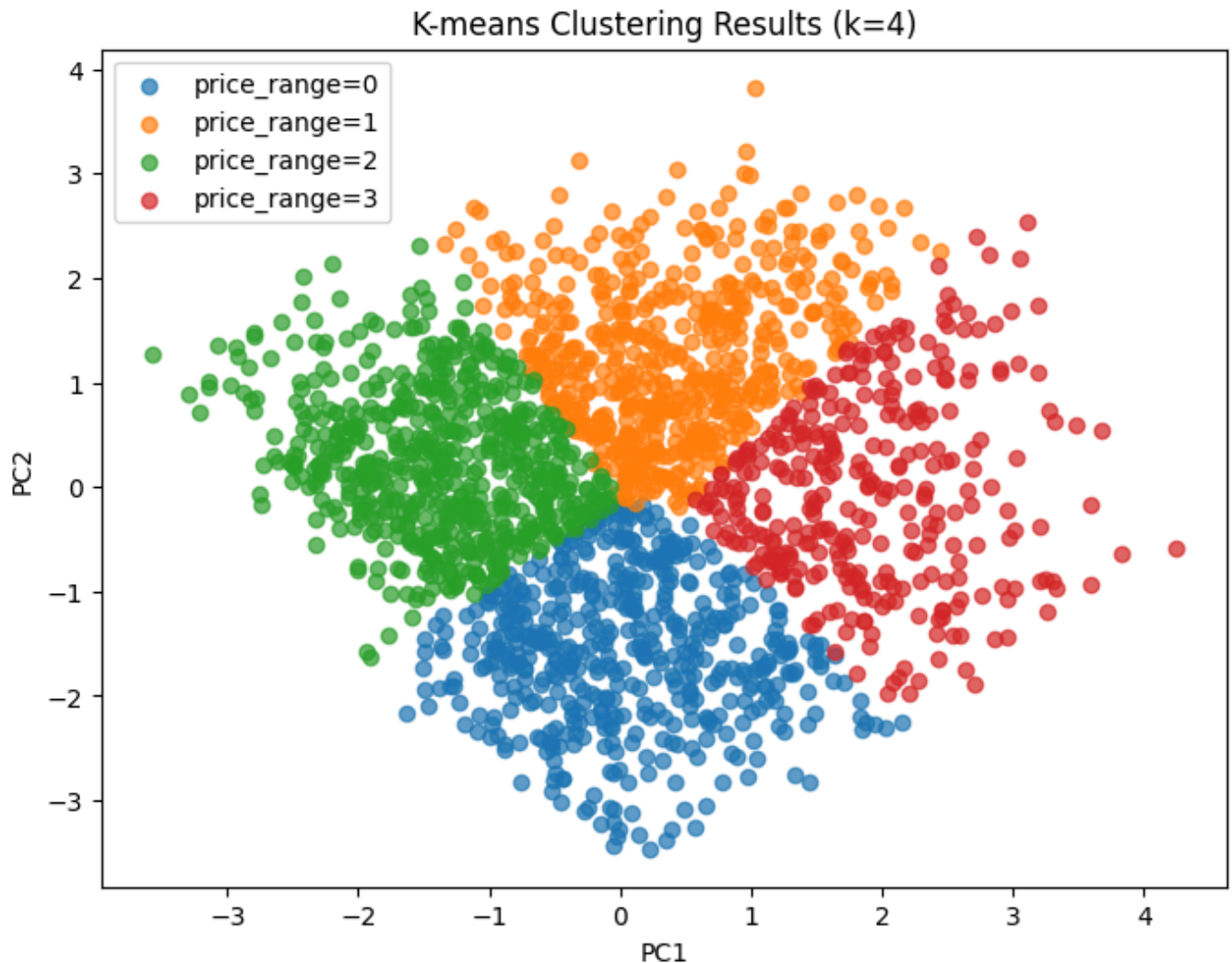


- c. Following 4(a), apply K-means to cluster the data samples into 4 clusters **using all features**. Visualize the clustering results in a scatter plot (including a legend), and report the clustering performance.



Adjusted Rand Score (ARS): 0.006067417030597427

- d. Following 4(b), apply K-means to cluster the data samples into 4 clusters using the **2-dimensional features obtained from PCA**. Visualize the clustering results with a scatter plot (including a legend), and report the clustering performance.



Adjusted Rand Score (ARS): 0.00043825468532816857

e. Please describe your observations in 4(c) and 4(d).

The K-means algorithm has a terrible score for this dataset (less than 0.007).

Using only the 2 principal components with PCA doesn't help at all; the score even decreased. The PCA projection of mobile prices in question 4(a) actually gives us the foreshadowing of this result: we cannot see a clear separation / pattern of the price ranges through these 2 parameters.

5. Enhancing K-means Clustering with Association Rule Mining (35 points).

Please propose a method to improve the performance of **K-means** using **association rule mining**, and implement your approach using all features in the data ("mobile_price.csv").

Clearly describe your design in detail and also your inspirations (e.g., inspired by which techniques, papers, or data observations), and compare the performance of the original K-means and your improved method in terms of accuracy, precision, recall, and F1-score.

To ensure the validity of your results, please report the **average performance of both the original K-means and your method** using the same set of random seeds by setting the `random_state` in K-means:

$$\text{seeds} = [0, 10, 42, 100, 999].$$

Also, you are free to draw your designed framework and provide any other more detailed analysis results.

Since we are only to use a modified version of KMeans, we don't have that much freedom, and one of the few things that we can change is the original central point of each cluster.

So, I had the idea to use the relevant transaction rules that are useful and use these relevant rules to set the initial center points.

I transformed each discrete value into three ranges, and instead of saving the transaction with the text description (low, medium, and high), I directly use the mean value of each range. Here are the discrete values:

```
['battery_power', 'clock_speed', 'fc', 'int_memory', 'm_dep', 'mobile_wt',  
'n_cores', 'pc', 'px_height', 'px_width', 'ram', 'sc_h', 'sc_w', 'talk_time']
```

I used all the features and listed all the association rules that are useful for price range prediction: (feature => price_range_i) where support ≥ 0.1 , confidence ≥ 0.25 , and lift ≥ 1 .

Then, I listed the most relevant association rules for each price range:

```
price_range_0  
ram      -> ram 815.579      (confidence=0.763, lift=3.053)  
battery_power -> battery_power_720.43 (confidence=0.332, lift=1.329)  
px height -> px height 292.318 (confidence=0.298, lift=1.191)  
sc_w     -> sc_w 8.59623    (confidence=0.277, lift=1.109)
```

px_width	-> px_width 1247.61	(confidence=0.264, lift=1.055)
touch_screen	-> touch_screen_1	(confidence=0.260, lift=1.042)
wifi	-> wifi_0	(confidence=0.256, lift=1.022)
dual_sim	-> dual_sim_0	(confidence=0.255, lift=1.019)
blue	-> blue_0	(confidence=0.254, lift=1.018)
fc	-> fc_1.74815	(confidence=0.255, lift=1.019)
clock_speed	-> clock_speed 0.692934	(confidence=0.254, lift=1.016)
four_g	-> four_g_0	(confidence=0.252, lift=1.007)

price_range_1

ram	-> ram 2135.69	(confidence=0.448, lift=1.793)
n_cores	-> n_cores 2.00544	(confidence=0.278, lift=1.110)
px_height	-> px_height 914.253	(confidence=0.263, lift=1.053)
clock_speed	-> clock_speed 0.692934	(confidence=0.263, lift=1.054)
int_memory	-> int_memory 33.0817	(confidence=0.263, lift=1.052)
sc_w	-> sc_w 2.4375	(confidence=0.268, lift=1.074)
battery_power	-> battery_power 1246.37	(confidence=0.270, lift=1.080)
touch_screen	-> touch_screen_1	(confidence=0.259, lift=1.038)
blue	-> blue_0	(confidence=0.252, lift=1.010)
fc	-> fc_1.74815	(confidence=0.253, lift=1.010)
wifi	-> wifi_0	(confidence=0.252, lift=1.006)
dual_sim	-> dual_sim_1	(confidence=0.250, lift=1.001)
four_g	-> four_g_1	(confidence=0.251, lift=1.005)

price_range_2

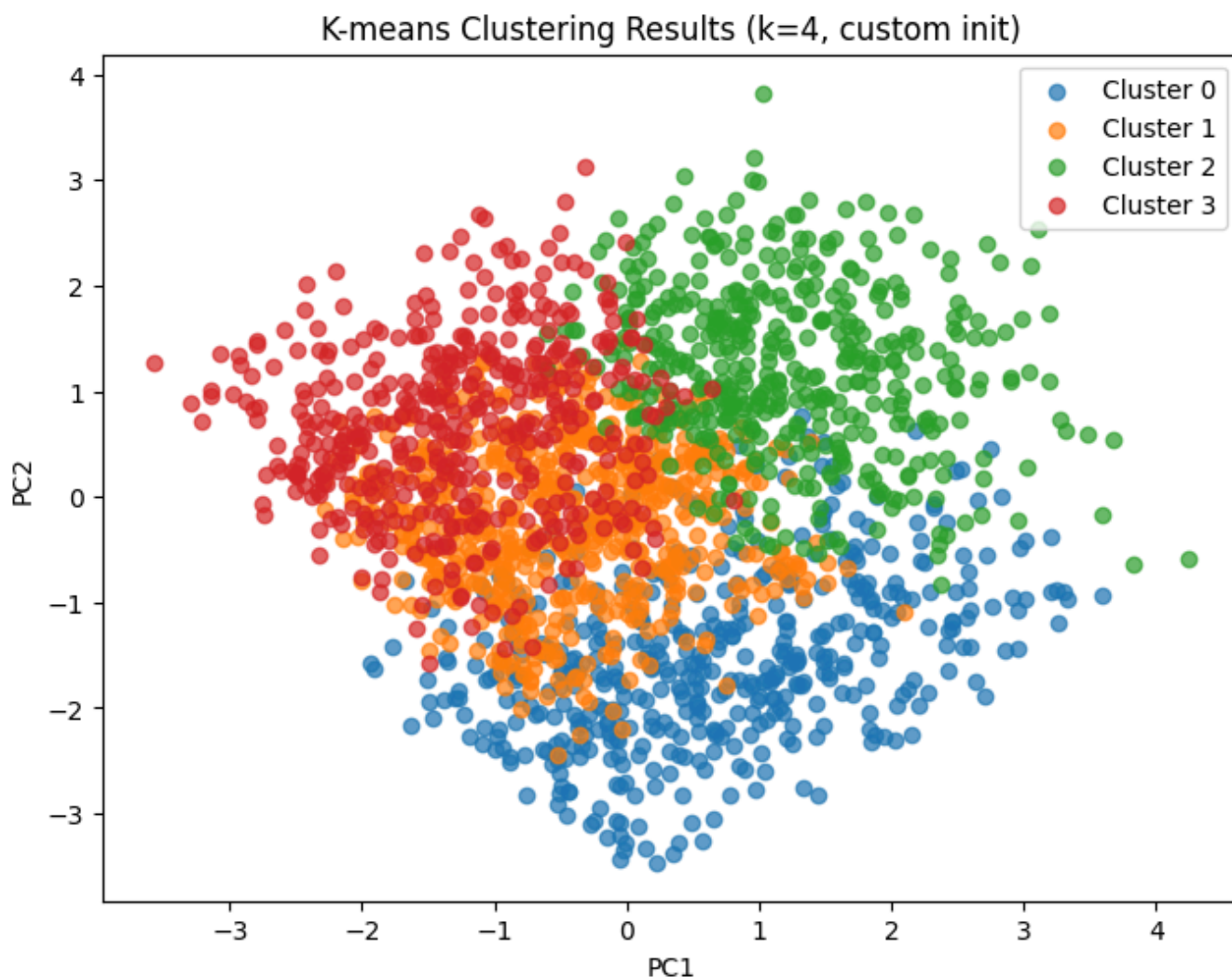
ram	-> ram 2135.69	(confidence=0.442, lift=1.768)
touch_screen	-> touch_screen_0	(confidence=0.267, lift=1.066)
mobile_wt	-> mobile_wt 140.769	(confidence=0.266, lift=1.065)
four_g	-> four_g_0	(confidence=0.264, lift=1.057)
px_height	-> px_height 292.318	(confidence=0.261, lift=1.044)
dual_sim	-> dual_sim_0	(confidence=0.256, lift=1.023)
blue	-> blue_0	(confidence=0.254, lift=1.018)
three_g	-> three_g_1	(confidence=0.254, lift=1.016)
wifi	-> wifi_0	(confidence=0.252, lift=1.006)
sc_w	-> sc_w 2.4375	(confidence=0.251, lift=1.004)

price_range_3

ram	-> ram 3457.18	(confidence=0.768, lift=3.072)
battery_power	-> battery_power 1773.31	(confidence=0.348, lift=1.391)
px_width	-> px_width 1779.72	(confidence=0.347, lift=1.389)
px_height	-> px_height 914.253	(confidence=0.278, lift=1.111)
int_memory	-> int_memory 33.0817	(confidence=0.266, lift=1.063)
four_g	-> four_g_1	(confidence=0.264, lift=1.055)
blue	-> blue_1	(confidence=0.262, lift=1.046)
dual_sim	-> dual_sim_1	(confidence=0.260, lift=1.040)
wifi	-> wifi_1	(confidence=0.258, lift=1.034)
touch_screen	-> touch_screen_0	(confidence=0.254, lift=1.014)
three_g	-> three_g_1	(confidence=0.253, lift=1.011)
fc	-> fc_1.74815	(confidence=0.252, lift=1.007)

Then I decided to use these values to set the initial center.

Here is the result for seed 42:



Adjusted Rand Score (ARS): 0.005520453922284243

(the result actually worsened...but:)

Results on different seeds:

```
Seed 0: default ARI = 0.0016, custom ARI = 0.0059
Seed 10: default ARI = 0.0017, custom ARI = 0.0069
Seed 42: default ARI = 0.0061, custom ARI = 0.0055
Seed 100: default ARI = 0.0035, custom ARI = 0.0061
Seed 999: default ARI = 0.0058, custom ARI = 0.0032
```

Average results

Default KMeans average ARI: 0.0038

Custom init average ARI: 0.0055

Standard deviation

Default KMeans std ARI: 0.0019

Custom init std ARI: 0.0012

My method has an average better score than the vanilla one.